Experimentação Florestal
(FL03034 - EF)

Introdução à Linguagem R

(Teoria e Prática)

Prof. Dr. Deivison Venicio Souza
Universidade Federal do Pará (UFPA)
Faculdade de Engenharia Florestal
Laboratório de Manejo Florestal, Tecnologias e Comunidades Amazônicas
E-mail: deivisonvs@ufpa.br

1ª versão: 25/agosto/2021
(Atualizado em: 20/outubro/2026)
Altamira, Pará

Objetivos

Ao final desta aula espera-se que o discente seja capaz de...

- Entender a importância e reconhecer a praticidade de uso de linguagem de programação;
- Realizar o download e a instalação do RGui e IDE Rstudio;
- Compreender as principais estruturas de dados na linguagem R e aprender a criá-las usando as funções do R-base;
- Realizar indexação de diferentes estruturas de dados no R; e
- Compreender e desenvolver suas próprias funções no R.

Conteúdo

Parte 1 - Conhecendo o R e o IDE Rstudio

1- A linguagem R

2 - Por que usar a linguagem R?

3 - RGui - Download, instalação e interface

4 - IDE RStudio - Download, instalação e interface

Parte 2 - Estrutura de dados na linguagem R

1 - Vetores

2 - Matrizes

3 - Data Frames

4 - Listas

Conteúdo

Parte 3 - Indexação no R

1 - Como fazer indexação no R?

2 - Indexação de vetores

3 - Indexação de matrizes

4 - Indexação de data frames

5 - Indexação de listas

Parte 4 - Um pouco mais sobre o R

1 - Sintaxe da linguagem R

2 - Operadores no R

- 2.1 - Aritméticos
- 2.2 - Relacionais
- 2.3 - Lógicos

3 - Funções matemáticas usuais

Parte 1

Conhecendo o R e o IDE Rstudio



Linguagem R

- É uma linguagem de programação de computadores de código aberto e gratuita;
- Criada em 1993: Ross Ihaka e por Robert Gentleman;
- Departamento de Estatística da Universidade de Auckland, Nova Zelândia;
- O R foi desenvolvido a partir da Linguagem S.

Possui múltiplas facetas:

- Pacotes para implementação de métodos estatisticos;
- Pacotes para visualização gráfica elegantes;
- Pacotes para criação de aplicações web, Dashboard;
- Pacote para geração de relatórios dinâmicos;
- Pacotes para criação de apresentações elegantes.

Por que usar a linguagem R?



RGui - Download e instalação

1º Passo: Acessar a página do projeto R em: https://www.r-project.org/;

2º Passo: Do lado esquerdo da página clique sobre o menu CRAN;

3º Passo: Será aberta uma página com diversos links de CRAN Mirrors, isto é, espelhos CRAN. Veja na tabela a seguir os principais espelhos disponíveis no Brasil.

Link	Instituição
http://cran-r.c3sl.ufpr.br/	Universidade Federal do Paraná - UFPR
http://nbcgib.uesc.br/mirrors/cran/	Center for Comp Biol at Universidade Estadual de Santa Cruz
https://cran.fiocruz.br/	Oswaldo Cruz Foundation, Rio de Janeiro
https://vps.fmvz.usp.br/CRAN/	University of São Paulo, São Paulo
http://brieger.esalq.usp.br/CRAN/	University of São Paulo, Piracicaba

Showing 1 to 5 of 5 entries Previous 1 Next

RGui - Download e instalação

4º Passo: Na página http://cran-r.c3sl.ufpr.br/, na seção Download and Install R, clicar em um dos três links, conforme o Sistema Operacional do usuário:

1. Download R for Windows;
2. Download R for Linux; ou
3. Download R for MacOS.

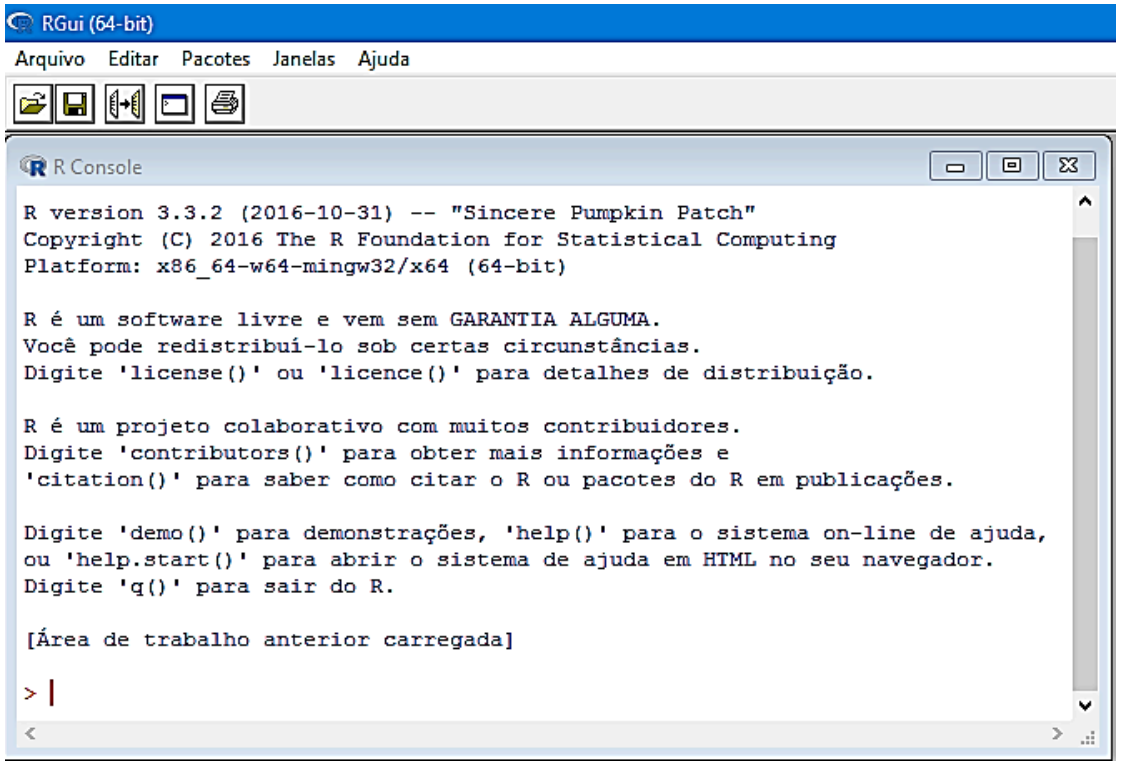
5º Passo: Clicar no link do subdiretório base ou em o install R for the first time para instalar o R pela primeira vez;

6º Passo: Clicar, por exemplo, em Download R-4.1.0. for Windows (escolha seu SO). Assim, será iniciado o download do R Development Core Team para o respectivo sistema; e

7º Passo: Por fim, basta usar o setup baixado para instalar o programa.

RGui - Interface

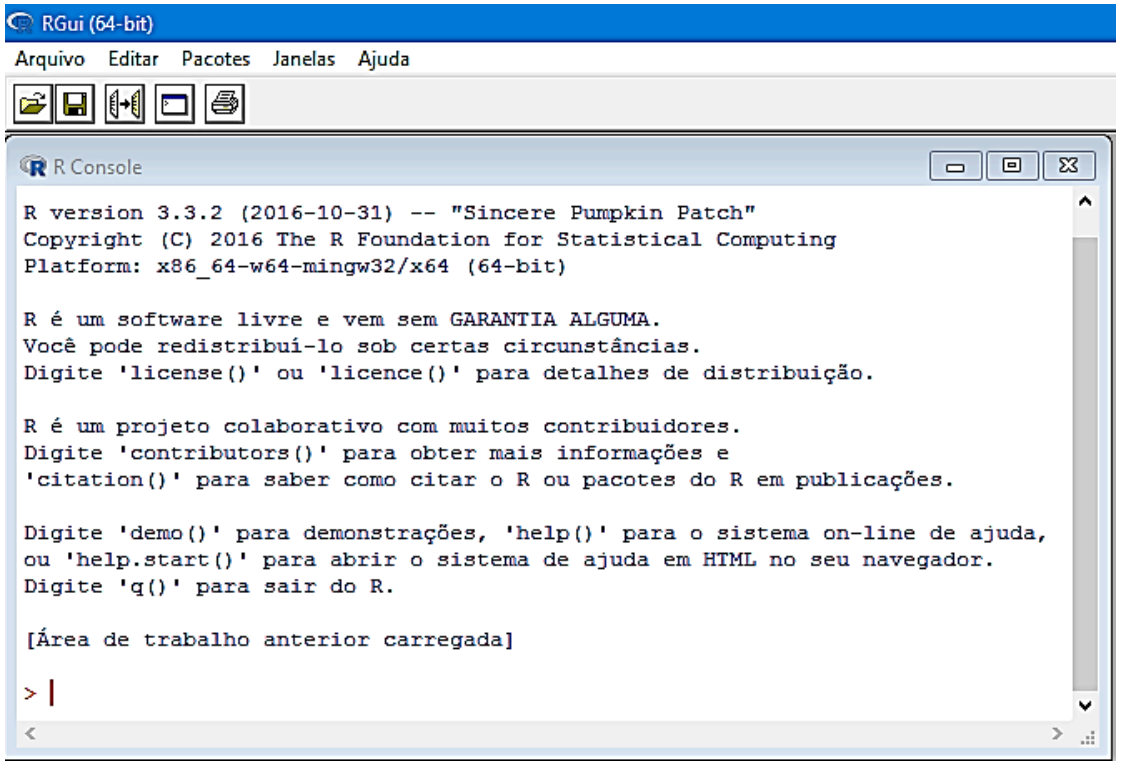
Ao inicializar o **R Development Core Team** pela primeira vez aparecerá a seguinte imagem:



R Console

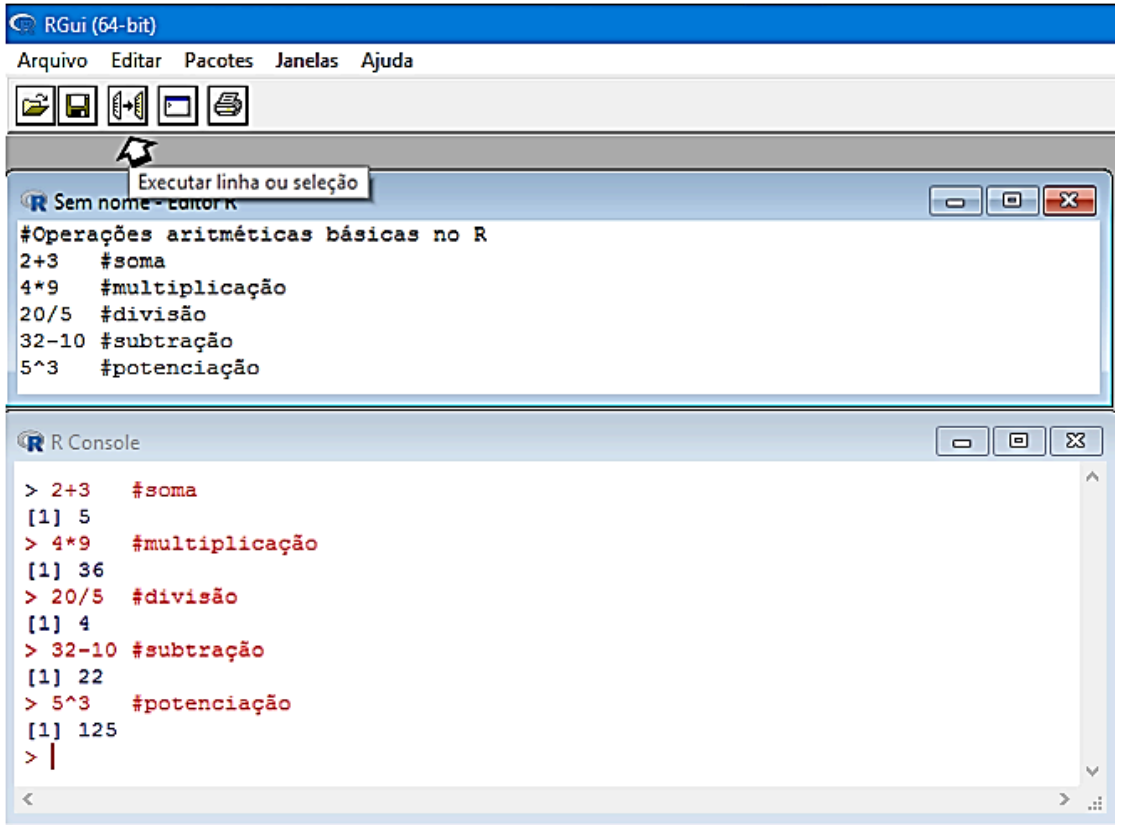
RGui - Interface

- No contato inicial do usuário com o RGui tem-se a visão do R Console.
- O sinal > é o prompt de comando.
- Execute as funções demo(), help(), help.start(), q().



RGui - R editor

O RGui possui um R editor → Abrir script.



R editor

IDE RStudio - download e instalação

O RStudio é um ambiente de desenvolvimento integrado (*Integrated Development Environment - IDE*) de códigos em R mais comumente usado por usuários da linguagem.

1º Passo: Acessar a página do projeto RStudio: https://www.rstudio.com;

2º Passo: Products $\rightarrow$ RStudio;

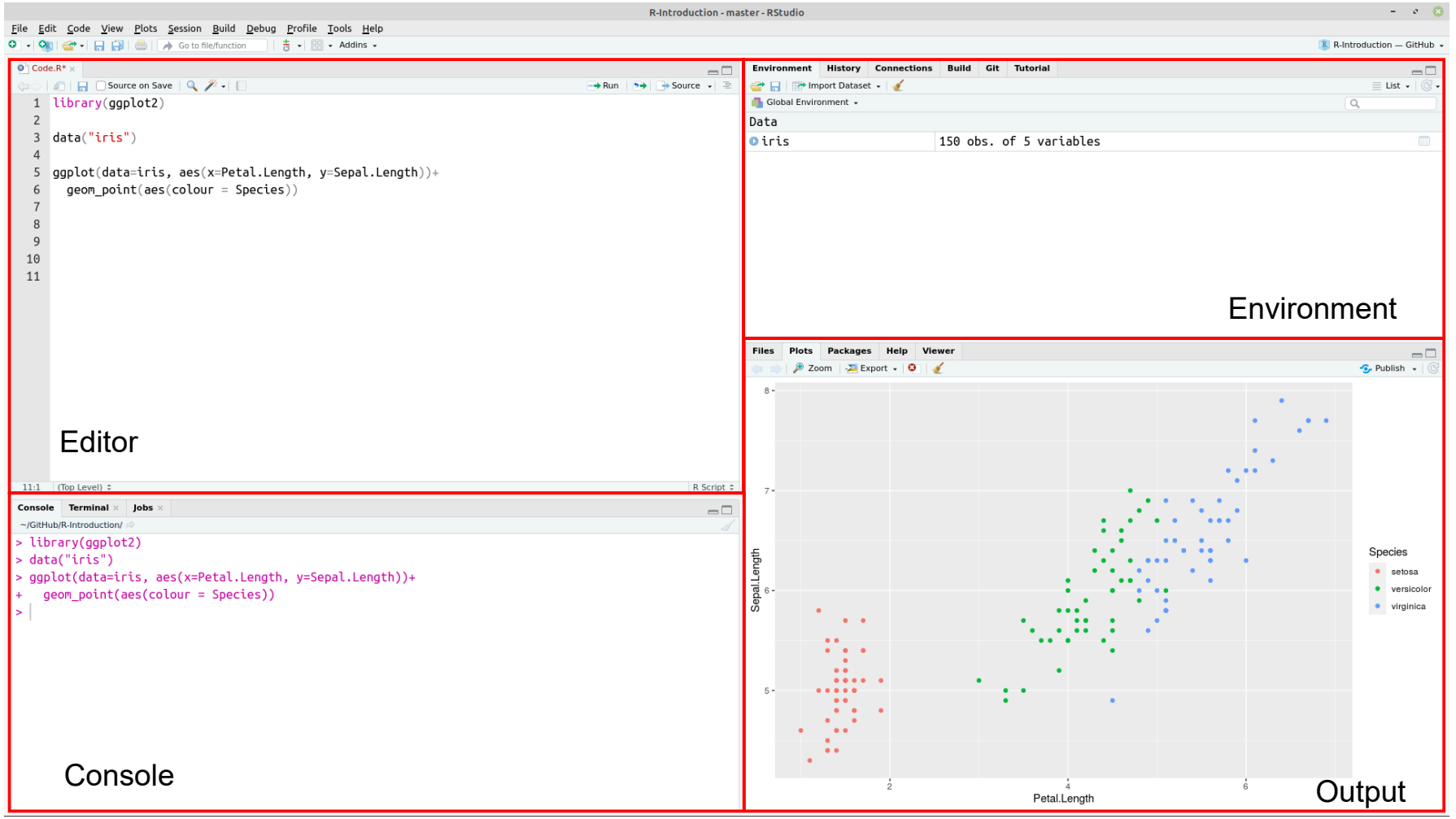
3º Passo: Selecionar a versão do RStudio para Desktop;

4º Passo: Na edição Open source $\rightarrow$ Download Rstudio Desktop;

5º Passo: Por fim, basta usar o setup baixado para instalar o programa.



IDE RStudio - Interface



IDE RStudio - Interface (Painéis)

Editor: Painel de desenvolvimento dos códigos R.

Environment: Painel onde aparecerão todos os objetos criados no R.

Console: Painel para rodar os códigos R e receber outputs.

Plots: Painel de saída gráfica.

History: Painel que mostra um histórico dos comandos executados na sessão corrente.

Help: Painel que mostra a documentação de funções de pacotes, quando solicitada.

Files: Painel para identificar arquivos no diretório de trabalho.

Packages: Painel que mostra os pacotes instalados. É possível identificar os pacotes carregados na sessão corrente.

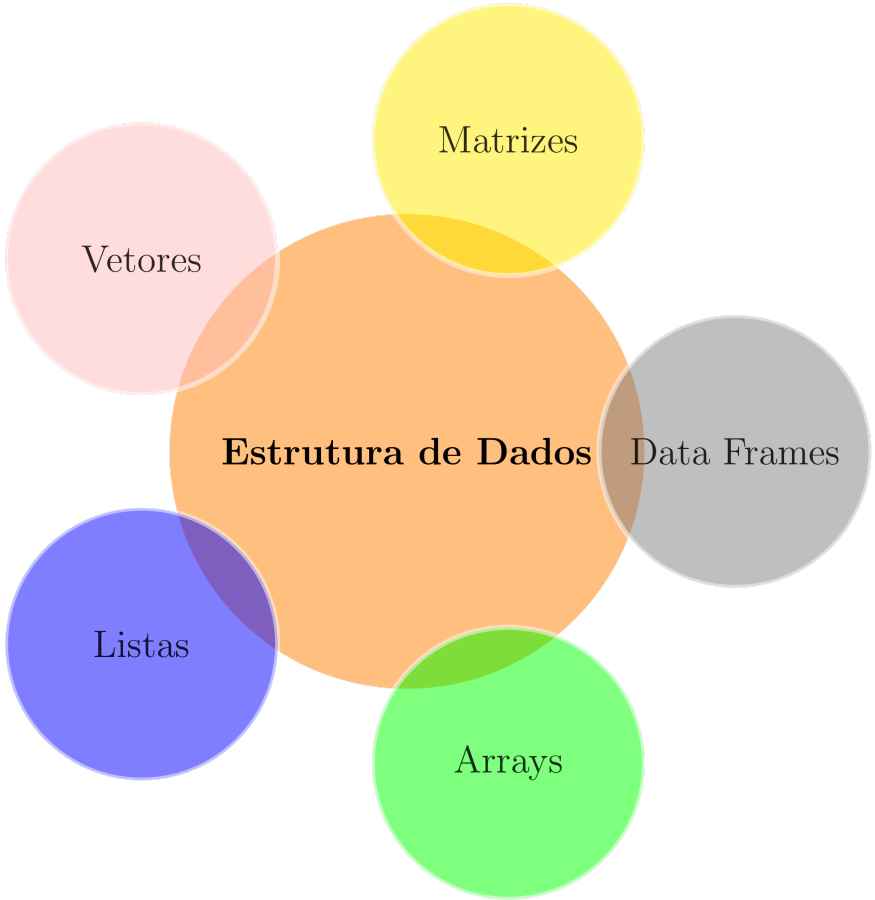
Parte 2

Estrutura de dados na linguagem R



Estrutura de dados na linguagem R

- O R pode armazenar dados em diferentes estruturas.



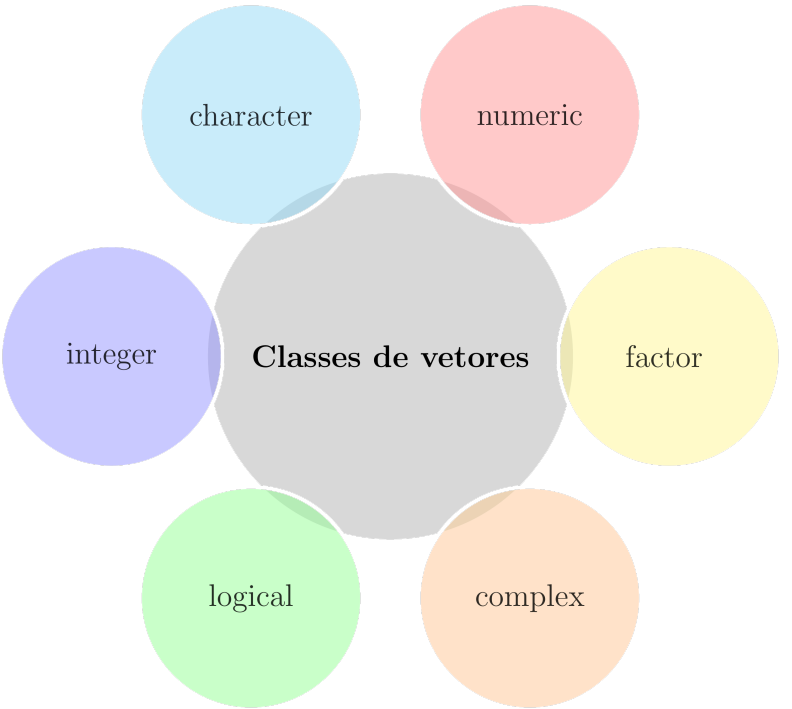
<!--

Estrutura de dados na linguagem R

Vetores

- É a forma mais simples de armazenar dados no R;
- São caracterizados por possuírem somente uma dimensão;
- Todos os seus elementos constituintes devem ter, obrigatoriamente, a mesma natureza (classe).

Principais classes de vetores



Estrutura de dados na linguagem R

Vetores - Principais funções do R-base para criar vetores...

1. Função c() (concatenate)

Função genérica que permite concatenar (combinar) argumentos para formar um vetor.

2. Função seq() (sequence)

Função genérica usada para gerar sequências de números em intervalos pré-definidos.

3. Função rep() (replicate)

Função genérica usada para replicar um elemento “x”.

4. Função scan()

Usada para criar vetores diretamente no R Console.

Estrutura de dados na linguagem R

Vetores - Função c() (concatenate)

Classe "character"

```
# Cria um vetor
c("Acapu", "Araucaria", "Mogno", "Cedro")
```

```
# Cria um vetor e faz atribuição
especie <- c("Acapu", "Mogno", "Cedro", "Ipe")
#class(especie)
```

Classe "numeric"

```
# Cria um vetor
c(23.0, 27.0, 33.6, 42.6, 52.1)
```

```
# Cria um vetor e faz atribuição
diametro <- c(23.0, 27.0, 33.6, 42.6, 52.1)
#class(diametro)
```

Estrutura de dados na linguagem R

Vetores - Função c() (concatenate)

Classe "factor"

- Por padrão, ao concatenar **strings** cria-se um vetor de classe **character**.
- Se desejável, pode-se codificar um vetor de classe **character** para a classe **factor**.
- O R-base possui duas funções: factor() e as.factor().
- Motivos: modelos estatísticos e gráficos

```
# Cria um vetor - Fitofisionomia
(Fito <- c("FOM", "FOA", "FOM", "FOM", "FOM", "FOA"))
## [1] "FOM" "FOA" "FOM" "FOM" "FOM" "FOA"
class(Fito)
## [1] "character"
```

```
# Codifica para "factor"
Fito <- c("FOM", "FOA", "FOM", "FOM", "FOM", "FOA")
(Fito <- factor(Fito))
## [1] FOM FOA FOM FOM FOM FOA
## Levels: FOA FOM
class(Fito)
## [1] "factor"
```

Vetores - Função c() (concatenate)

Classe "factor" - declarando hierarquia

- Por padrão, os níveis (levels) de um vetor de classe **factor** ficam dispostos em ordem alfabética.
- Em algumas situações, pode ser desejável estabelecer uma hierarquia entre esses níveis.
- Por exemplo, imagine a variável mês de coleta. É evidente que existe hierarquia entre os meses.

Níveis do mês não ordenados

```
(mes <- factor(
  c("Janeiro", "Abril", "Julho", "Outubro")
  ))
## [1] Janeiro Abril   Julho   Outubro
## Levels: Abril Janeiro Julho Outubro
```

Níveis do mês ordenados

```
(mes <-
   factor(x = mes,
          levels = c("Janeiro", "Abril",
                     "Julho", "Outubro"),
                ordered = TRUE))
## [1] Janeiro Abril   Julho   Outubro
## Levels: Janeiro < Abril < Julho < Outubro
```

Estrutura de dados na linguagem R

Vetores - Função seq() (sequence)

seq(**from** = ?, **to** = ?, **by** = ?, **length.out** = ?, **along.with** = ?)

from = define um valor inicial (start) para a sequência.

to = define um valor máximo para a sequência.

by = define um valor incremental para a sequência.

length.out = define o comprimento da sequência dentro de um intervalo.

along.with = define uma sequência de inteiros do tamanho do argumento repassado.

Estrutura de dados na linguagem R

Vetores - Função seq() (sequence)

```r
# 1 a 10, com intervalo 1.
seq(10)

# Sequência de 1 a 10, com intervalo 1.
seq(1:10)

# Sequência de 1 a 10, com intervalo 1.
seq(from = 1, to = 10, by = 1)

# Sequência de -2 a 10, com intervalo 2.
seq(from = -2, to = 10, by = 2)

# Sequência de 11 números entre 0 e 1.
seq(from=0, to=1, length.out = 5)

# Sequência do tamanho do vetor-espécie.
especie <- c("Acapu", "Mogno", "Cedro", "Ipe")
seq(along.with = especie)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
##  [1]  1  2  3  4  5  6  7  8  9 10
##  [1]  1  2  3  4  5  6  7  8  9 10
## [1] -2  0  2  4  6  8 10
## [1] 0.00 0.25 0.50 0.75 1.00
## [1] 1 2 3 4
```

Estrutura de dados na linguagem R

Vetores - Função rep() (replicate)

rep(**x** = ?, **times** = ?, **each** = ?)

x = escalar, vetor ou uma lista, entre outros.

times = um vetor de inteiro (não negativo) indicando quantas vezes repetir cada elemento.

each = cada elemento de x é repetido n vezes.

Estrutura de dados na linguagem R

Vetores - Função rep() (replicate)

```r
# Repete a sequência de 1 a 4 (2x).
rep(x = 1:4, 2)

# Repete a sequência de 1 a 4 (3x).
rep(x = 1:4, times = 3)

# Repete cada valor em "x" (2x).
rep(x = 1:4, each = 2)

# Repete cada valor em "x" (2x).
rep(x = 1:4, c(2,2,2,2))

# Repete cada valor em "x", com base em c().
rep(1:4, c(2,1,2,1))

# Repete cada valor em "x", até 4 números.
rep(1:4, each = 2, len = 4)

# Repete cada valor em "x" (2x), por 3x.
rep(1:2, each = 2, times = 3)
```

```
## [1] 1 2 3 4 1 2 3 4
##  [1] 1 2 3 4 1 2 3 4 1 2 3 4
## [1] 1 1 2 2 3 3 4 4
## [1] 1 1 2 2 3 3 4 4
## [1] 1 1 2 3 3 4
## [1] 1 1 2 2
##  [1] 1 1 2 2 1 1 2 2 1 1 2 2
```

Estrutura de dados na linguagem R

Vetores - Função scan()

Procedimento

Passo 1º: digitar no R console, por exemplo: y <- scan()

Passo 2º: Digitar o primeiro dado e apertar $< ENTER >$. Seguir esse passo até o último dado.

Passo 3º: Para encerrar a entrada de dados basta apertar $< ENTER >$ duas vezes.

Atividade para 🏠

Use a função scan() para criar vetores para as variáveis da tabela a seguir diretamente no R console:

```
##     especie diametro altura
## 1     Acapu     23.0    8.5
## 2 Araucaria     27.0    9.2
## 3     Mogno     33.6   10.5
## 4     Cedro     42.6   13.4
## 5       Ipe     52.1   15.8
```

Obs.: Para vetores da class **character** especifique o argumento **what = character()**.

y <- scan(what = character())

Estrutura de dados na linguagem R

Matrizes - Propriedades

- São caracterizadas por possuírem duas dimensões (linhas e colunas) - bidimensional.
- Todos os seus elementos possuem a mesma classe (numeric, character ou logical).
- O R-base possui 3 funções principais: cbind(), rbind(), matrix().
- A função matrix() é considerada mais eficiente.

matrix(**data** = NA, **nrow** = 1, **ncol** = 1, **byrow** = FALSE, **dimnames** = NULL)

data = um vetor de dados (opcional).

nrow = número de linhas da matriz.

ncol = número de colunas da matriz.

byrow = argumento lógico. Se **FALSE** (padrão), a matriz é preenchida por colunas. Se **TRUE**, a matriz é preenchida por linhas.

dimnames = Insere os rótulos das linhas e colunas da matriz com a função list().

Estrutura de dados na linguagem R

Matrizes - Função matrix()

```
# Cria uma matriz de 1 a 6, com 2 linhas e 3 colunas
(mat1 <- matrix(data=1:6, nrow=2, ncol=3))
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
# Atribui nomes às linhas e colunas
rownames(mat1) <- c("L1","L2"); print(mat1)

colnames(mat1) <- c("C1","C2","C3"); print(mat1)
```

```
##    [,1] [,2] [,3]
## L1    1    3    5
## L2    2    4    6
```

```
##    C1 C2 C3
## L1  1  3  5
## L2  2  4  6
```

- Se desejável, pode-se usar as funções rownames() e colnames() para atribuir nomes às linhas e colunas da matriz.

Estrutura de dados na linguagem R

Matrizes - Função matrix()

- Outra forma de atribuir nomes às linhas e colunas é usar o parâmetro **dimnames** da própria função matrix().

```
(mat2 <-
   matrix(data=1:6, nrow=2,
          ncol=3, byrow=TRUE,
          dimnames=list(c("L1", "L2"),
                        c("C1", "C2", "C3"))))
```

```
##    C1 C2 C3
## L1  1  2  3
## L2  4  5  6
```

Estrutura de dados na linguagem R

Matrizes - Situações especiais

- Ao criar-se matrizes atentar-se para duas situações:

1 - **Descarte de elementos**: Quando a quantidade de elementos (n) for maior que a quantidade de linhas e colunas (ncol x nrow).

2 - **Regra da Reciclagem**: Quando a quantidade de linhas e colunas (ncol x nrow) for maior do que a quantidade de elementos (n).

Estrutura de dados na linguagem R

Data Frames - Propriedades

- São caracterizados por possuírem duas dimensões (linhas e colunas) - bidimensional.
- Pode-se reunir vetores de diferentes classes, com a condição de possuírem igual comprimento.
- O R-base possui a função data.frame().
- A edição de um data.frame pode ser feita usando a função edit().
- Em geral, dados importados (.csv e .txt) são armazenados como data.frame.

Atividade para 🏠

Use o comando x <- edit(data.frame()) e crie a tabela a seguir:

```
##     especie diametro altura
## 1     Acapu     23.0    8.5
## 2 Araucaria     27.0    9.2
## 3     Mogno     33.6   10.5
## 4     Cedro     42.6   13.4
## 5       Ipe     52.1   15.8
```

Estrutura de dados na linguagem R

Data Frames - Função data.frame()

Cria um data frame a partir de vetores existentes

```
especie <- c("Acapu", "Araucaria", "Cedro", "Tauari")
diametro <- c(23.0, 27.0, 33.6, 52.6)
altura <- c(8.4, 8.7, 9.1, 18.2)
cortar <- c("Não", "Não", "Não", "Sim")

(invFlor1 <-
  data.frame(especie, diametro, altura, cortar,
             stringsAsFactors = TRUE))
```

```
##     especie diametro altura cortar
## 1     Acapu     23.0    8.4    Não
## 2 Araucaria     27.0    8.7    Não
## 3     Cedro     33.6    9.1    Não
## 4    Tauari     52.6   18.2    Sim
```

Cria um data frame com apenas um comando

```
(invFlor2 <-
  data.frame(
    especie=c("Acapu", "Araucaria", "Cedro", "Tauari"
    diametro=c(23.0, 27.0, 33.6, 52.6),
    altura=c(8.4, 8.7, 9.1, 18.2),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE))
```

```
##     especie diametro altura cortar
## 1     Acapu     23.0    8.4    Não
## 2 Araucaria     27.0    8.7    Não
## 3     Cedro     33.6    9.1    Não
## 4    Tauari     52.6   18.2    Sim
```

Explore as funções dim() e str().

Estrutura de dados na linguagem R

Listas - Propriedades

- Pode reunir diferentes estruturas de dados (vetores, matrizes, data frames e inclusive outras listas).
- O R-base possui a função list().
- Se os objetos já existem pode-se simplesmente adicioná-los à função list().

Estrutura de dados na linguagem R

Listas - Função list()

Uma lista a partir de objetos já existentes

```
especie <- c("Acapu", "Mogno", "Cedro", "Ipe")
diametro <- c(23.0, 27.0, 33.6, 52.6)
altura <- c(8.4, 8.7, 9.1, 18.2)

# Sem atribuir nomes aos objetos da lista
(list1 <- list(especie, diametro, altura))

# Atribuindo nomes aos objetos da lista
(list2 <- list(Esp = especie,
               DAP = diametro,
               H = altura))
```

```
## [[1]]
## [1] "Acapu" "Mogno" "Cedro" "Ipe"
##
## [[2]]
## [1] 23.0 27.0 33.6 52.6
##
## [[3]]
## [1]  8.4  8.7  9.1 18.2
```

```
## $Esp
## [1] "Acapu" "Mogno" "Cedro" "Ipe"
##
## $DAP
## [1] 23.0 27.0 33.6 52.6
##
## $H
## [1]  8.4  8.7  9.1 18.2
```

Nomear os componentes da lista facilita a identificação e a indexação.

Estrutura de dados na linguagem R

Listas - Função list()

Uma lista com diferentes estruturas de dados

```
diametro <- c(23.0, 27.0, 33.6, 52.6)

mat <-
   matrix(data=1:6, nrow=2,
          ncol=3, byrow=TRUE,
          dimnames=list(c("L1", "L2"),
                        c("C1", "C2", "C3")))

invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Ipe"),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE)

# Criando lista nomeada
(list3 <- list(Vetor = diametro,
               Matriz = mat,
               DataFrame = invFlor))
```

```
## $Vetor
## [1] 23.0 27.0 33.6 52.6
##
## $Matriz
##    C1 C2 C3
## L1  1  2  3
## L2  4  5  6
##
## $DataFrame
##   especie cortar
## 1   Acapu    Não
## 2   Mogno    Não
## 3   Cedro    Não
## 4     Ipe    Sim
```

Explore as funções length() e names().

Estrutura de dados na linguagem R

Listas - Coagindo lista para data frame

- Uma lista pode ser transformada em um data frame usando a função as.data.frame().
- Mas, para isso a lista a ser transformada deve possuir vetores de igual comprimento.

Lista para Data Frame

```
# Criando lista nomeada
list4 <- list(
  Esp = c("Acapu", "Mogno", "Cedro", "Ipe"),
  DAP = c(23.0, 27.0, 33.6, 52.6),
  H = c(8.4, 8.7, 9.1, 18.2))

as.data.frame(list4)
```

```
##     Esp  DAP    H
## 1 Acapu 23.0  8.4
## 2 Mogno 27.0  8.7
## 3 Cedro 33.6  9.1
## 4   Ipe 52.6 18.2
```

Parte 3

Indexação no R



Indexação no R

- Quando o interesse é **extrair**, **excluir** ou **substituir** elementos de objetos é possível fazê-lo por meio de algum mecanismo de indexação.
- Operadores de indexação: [ ], [[ ]] e $.
- O operador $: permite extrair componentes nomeados de uma lista ou data frame.

Indexação no R

Indexação de vetores

- Para extrair, excluir ou substituir elementos no objeto-vetor usa-se o comando [i].
- O índice i indica a posição do elemento no objeto, e inicia no valor 1.
- A função c() pode ser usada para concatenar as posições dentro de colchetes.

Indexação no R

Indexação de vetores - Extração

Extração por indexação positiva

```
# cria os vetores
especie <- c("Acapu", "Mogno", "Cedro", "Ipe")
diametro <- c(23.0, 27.0, 33.6, 42.6)

# Um elemento.
especie[2]

# Múltiplos elementos (sequenciais).
diametro[1:3]

# Múltiplos elementos (alternados).
especie[c(1,3,4)]

# Múltiplos elementos usando seq().
diametro[seq(from = 1, to = 4, by = 1)]
```

```
## [1] "Mogno"
```

```
## [1] 23.0 27.0 33.6
```

```
## [1] "Acapu" "Cedro" "Ipe"
```

```
## [1] 23.0 27.0 33.6 42.6
```

Indexação no R

Indexação de vetores - Extração

Extração por indexação negativa

```
# cria os vetores
especie <- c("Acapu", "Mogno", "Cedro", "Ipe")
diametro <- c(23.0, 27.0, 33.6, 42.6)

# Um elemento.
especie[-2]

# Múltiplos elementos (sequenciais).
diametro[-(1:3)]

# Múltiplos elementos (alternados).
especie[-c(1,3,5)]
```

```
## [1] "Acapu" "Cedro" "Ipe"
```

```
## [1] 42.6
```

```
## [1] "Mogno" "Ipe"
```

Indexação no R

Indexação de vetores - Extração

Extração por indexação lógica

```
Especie <- c("Mogno", "Cedro", "Ipe", "Tauari")
Diametro <- c(23.0, 27.0, 33.6, 42.6, 52.1)

# Extrai árvores que não sejam Tauari
Especie[Especie != "Tauari"]

# Diâmetros >= 50cm.
Diametro[Diametro >= 50]
```

```
## [1] "Mogno" "Cedro" "Ipe"
```

```
## [1] 52.1
```

Indexação no R

Indexação de vetores - Substituição

Extração por indexação lógica

```
Especie <- c("Mogno", "Cedro", "Ipe", NA, NA)
Diametro <- c(23.0, 27.0, 33.6, 42.6, 52.1)

# Substitui os NAs
Especie[is.na(Especie)] <- "NI"
print(Especie)

# Altera posição 3, e atribui 33.5.
Diametro[3] <- 33.5; print(Diametro)

# Altera posições 4 e 5, e atribui 55.3 e 63.4
Diametro[c(4, 5)] <- c(55.3, 63.4); print(Diametro)
```

```
## [1] "Mogno" "Cedro" "Ipe"   "NI"    "NI"
```

```
## [1] 23.0 27.0 33.5 42.6 52.1
```

```
## [1] 23.0 27.0 33.5 55.3 63.4
```

Indexação no R

Indexação de matrizes

- Para **extrair**, **excluir** ou **substituir** elementos de uma matriz usa-se o comando [i, j].
- O índice i indica as linhas e o índice j indica as colunas da matriz.
- Se o argumento for do tipo [i,] ter-se-á acesso a todos os elementos da linha i especificada.
- Se o argumento for do tipo [, j] ter-se-á acesso a todos os elementos da coluna j especificada.
- Se nem o número da linha e nem o número da coluna é informado [,]: a matriz é acessada por completa.

Indexação no R

Indexação de matrizes - Extração

Extração por indexação positiva

```
# Cria uma matriz
(mat <-
  matrix(1:6, nrow=2, ncol=3, byrow = TRUE,
         dimnames = list(c("L1", "L2"),
                         c("C1", "C2", "C3"))))


# extrai o elemento da linha 2 e coluna 2.
mat[2, 2]

# extrai todos elementos da linha 2.
mat[2, ]

# extrai todos elementos da coluna 3.
mat[, 3]

# extrai os elementos de L1 e L2, C2 e C3.
mat[c(1, 2), c(2, 3)]
```

```
##    C1 C2 C3
## L1  1  2  3
## L2  4  5  6
```

```
## [1] 5
```

```
## C1 C2 C3
##  4  5  6
```

```
## L1 L2
##  3  6
```

```
##    C2 C3
## L1  2  3
## L2  5  6
```

Indexação no R

Indexação de matrizes - Extração

Extração por indexação negativa

```
# Cria uma matriz
(mat <-
  matrix(1:6, nrow=2, ncol=3, byrow = TRUE,
         dimnames = list(c("L1", "L2"),
                         c("C1", "C2", "C3"))))

# exclui as colunas 1 e 3.
mat[, c(-1, -3)]

# exclui apenas a coluna 2
mat[, -2]
```

```
##    C1 C2 C3
## L1  1  2  3
## L2  4  5  6
```

```
## L1 L2
##  2  5
```

```
##    C1 C3
## L1  1  3
## L2  4  6
```

Indexação no R

Indexação de matrizes - Substituição

Substituição de elementos na matriz

```
# Cria uma matriz
(mat <-
  matrix(1:6, nrow=2, ncol=3, byrow = TRUE,
         dimnames = list(c("L1", "L2"),
                         c("C1", "C2", "C3"))))

# substitui o elemento da posição 1 por zero.
mat[1, 1] <- 0; print(mat)

# substitui os elementos das posições 1 e 5 por zero.
mat[c(1,5)] <- c(0,0); print(mat)
```

```
##    C1 C2 C3
## L1  1  2  3
## L2  4  5  6
```

```
##    C1 C2 C3
## L1  0  2  3
## L2  4  5  6
```

```
##    C1 C2 C3
## L1  0  2  0
## L2  4  5  6
```

Indexação no R

Indexação de data frames

- A **extração**, **inclusão** ou **substituição** de vetores em um data frame pode ser feita usando os comandos: [*i*, *j*] e $.
- O comando $ é usual para colunas nomeadas.
- O comando [*i*, *j*] segue a lógica de matrizes. Em que, *i* = linhas e *j* = colunas.
- Operadores **lógicos** (&, |, !) e de **comparação** (<, >, >=, <=, !=, ==, %in%) podem ser usados para extrair informações específicas do data frame.
- As funções attach() (attachment) e with() também podem ser usadas para facilitar o acesso à colunas de data frames.

Indexação no R

Indexação de data frames - Comando [*i*, *j*]

Extração de elementos do data frame

```
(invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Tauari"),
    diametro=c(39.5, 45.6, 49.5, 70.4),
    altura=c(10.5, 13.6, 14.5, 17.4),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE))

# Extrai elementos da linha 2 e coluna 1
invFlor[2, 1]

# Extrai todos os elementos da coluna 2
invFlor[, 2]

# Extrai todos os elementos da coluna 2
# Porém, conserve a saída como data frame
invFlor[, 2, drop=FALSE]
```

```
##   especie diametro altura cortar
## 1   Acapu     39.5   10.5    Não
## 2   Mogno     45.6   13.6    Não
## 3   Cedro     49.5   14.5    Não
## 4  Tauari     70.4   17.4    Sim
```

```
## [1] "Mogno"
```

```
## [1] 39.5 45.6 49.5 70.4
```

```
##   diametro
## 1     39.5
## 2     45.6
## 3     49.5
## 4     70.4
```

Indexação no R

Indexação de data frames - Comando [*i*, *j*]

Extração de elementos do data frame

```
(invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Tauari"),
    diametro=c(39.5, 45.6, 49.5, 70.4),
    altura=c(10.5, 13.6, 14.5, 17.4),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE))

# Extrai colunas pelo nome
invFlor[, "especie"]

# Extrai grupos de colunas
# Observe a ordem de impressão das variáveis
invFlor[, c(4,3)]
invFlor[, c("cortar", "especie")]
```

```
##   especie diametro altura cortar
## 1   Acapu     39.5   10.5    Não
## 2   Mogno     45.6   13.6    Não
## 3   Cedro     49.5   14.5    Não
## 4  Tauari     70.4   17.4    Sim
```

```
## [1] "Acapu"  "Mogno"  "Cedro"  "Tauari"
```

```
##   cortar altura
## 1    Não   10.5
## 2    Não   13.6
## 3    Não   14.5
## 4    Sim   17.4
```

```
##   cortar especie
## 1    Não   Acapu
## 2    Não   Mogno
## 3    Não   Cedro
## 4    Sim  Tauari
```

Indexação no R

Indexação de data frames - Comando $

Extração de elementos do data frame

```
(invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Tauari"),
    diametro=c(39.5, 45.6, 49.5, 70.4),
    altura=c(10.5, 13.6, 14.5, 17.4),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE))

# extrai a coluna “diâmetro”
invFlor$diametro

# Acessa a coluna “altura”
# e extrai os elementos da posição [4, 5]
invFlor$altura[c(3,4)]
```

```
##   especie diametro altura cortar
## 1   Acapu     39.5   10.5    Não
## 2   Mogno     45.6   13.6    Não
## 3   Cedro     49.5   14.5    Não
## 4  Tauari     70.4   17.4    Sim
```

```
## [1] 39.5 45.6 49.5 70.4
```

```
## [1] 14.5 17.4
```

Indexação no R

Indexação de data frames - $ + Operadores de comparação

Extração de elementos do data frame

```
(invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Tauari"),
    diametro=c(39.5, 45.6, 49.5, 70.4),
    altura=c(10.5, 13.6, 14.5, 17.4),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE))

# Quais árvores têm mais de 14 m?
invFlor$altura > 14

# Quais árvores têm diâmetros >= 50cm?
invFlor$diametro >= 50

# Quais árvores estão previstas para corte?
invFlor$cortar == "Sim"
```

```
##   especie diametro altura cortar
## 1   Acapu     39.5   10.5    Não
## 2   Mogno     45.6   13.6    Não
## 3   Cedro     49.5   14.5    Não
## 4  Tauari     70.4   17.4    Sim
```

```
## [1] FALSE FALSE  TRUE  TRUE
```

```
## [1] FALSE FALSE FALSE  TRUE
```

```
## [1] FALSE FALSE FALSE  TRUE
```

Observe que as saídas dos comandos são respostas booleanas (TRUE ou FALSE). No entanto, na maioria das vezes deseja-se extrair os elementos.

Indexação no R

Indexação de data frames - [*i*, *j*] + $ + Operadores (lógicos e comparação)

Extração de elementos do data frame

```
(invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Tauari"),
    diametro=c(39.5, 45.6, 49.5, 70.4),
    altura=c(10.5, 13.6, 14.5, 17.4),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE))

# Quais árvores têm mais de 14 m?
invFlor[invFlor$altura > 14, ]

# Quais árvores têm diâmetros >= 50cm?
invFlor[invFlor$diametro >= 50, ]

# Quais árvores estão previstas para corte?
invFlor[invFlor$cortar == "Sim", ]

# Quais árvores têm diametro < 50
# e não estão previstas para corte?
invFlor[invFlor$diametro < 50 & invFlor$cortar == "Não", ]
```

```
##   especie diametro altura cortar
## 1   Acapu     39.5   10.5    Não
## 2   Mogno     45.6   13.6    Não
## 3   Cedro     49.5   14.5    Não
## 4  Tauari     70.4   17.4    Sim
```

```
##   especie diametro altura cortar
## 3   Cedro     49.5   14.5    Não
## 4  Tauari     70.4   17.4    Sim
```

```
##   especie diametro altura cortar
## 4  Tauari     70.4   17.4    Sim
```

```
##   especie diametro altura cortar
## 4  Tauari     70.4   17.4    Sim
```

```
##   especie diametro altura cortar
## 1   Acapu     39.5   10.5    Não
## 2   Mogno     45.6   13.6    Não
## 3   Cedro     49.5   14.5    Não
```

Indexação no R

Indexação de data frames - Adição de linhas e colunas

```
(invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Tauari"),
    diametro=c(39.5, 45.6, 49.5, 70.4),
    altura=c(10.5, 13.6, 14.5, 17.4),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE))

# Cria um vetor sobre proteção
protegida <- c("Sim", "Sim", "Sim", "Não")

# Adiciona como nova coluna
invFlor$protegida <- protegida
print(invFlor)

# Substitui linha na posição 1
invFlor[1,] <- c("Acapu", 50.9, 15.6, "Não", "Sim")
print(invFlor)
```

```
##   especie diametro altura cortar
## 1   Acapu     39.5   10.5    Não
## 2   Mogno     45.6   13.6    Não
## 3   Cedro     49.5   14.5    Não
## 4  Tauari     70.4   17.4    Sim
```

```
##   especie diametro altura cortar protegida
## 1   Acapu     39.5   10.5    Não       Sim
## 2   Mogno     45.6   13.6    Não       Sim
## 3   Cedro     49.5   14.5    Não       Sim
## 4  Tauari     70.4   17.4    Sim       Não
```

```
##   especie diametro altura cortar protegida
## 1   Acapu     50.9   15.6    Não       Sim
## 2   Mogno     45.6   13.6    Não       Sim
## 3   Cedro     49.5   14.5    Não       Sim
## 4  Tauari     70.4   17.4    Sim       Não
```

Indexação no R

Indexação de listas

- A indexação de lista pode ser feita com uso dos comandos [[ ]] e $.
- Para acessar subíndices dos componentes da lista pode-se fazer: [[ ]][ ] .
- O comando $ poderá ser usado quando a lista tiver seus componentes nomeados.

Indexação no R

Indexação de listas - Níveis superiores não nomeados

Lista com níveis superiores não nomeados

```
diametro <- c(23.0, 27.0, 33.6, 52.6)

invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Tauari"),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE)

# Criando lista não nomeada
(list <- list(diametro, invFlor))

# extrai o componente 1
list[[1]]

# extrai a posição 1 do componente 2
list[[2]][1]
#list[[2]]["especie"]
list[[2]]$especie
```

```
## [[1]]
## [1] 23.0 27.0 33.6 52.6
##
## [[2]]
##   especie cortar
## 1   Acapu    Não
## 2   Mogno    Não
## 3   Cedro    Não
## 4  Tauari    Sim
```

```
## [1] 23.0 27.0 33.6 52.6
```

```
##   especie
## 1   Acapu
## 2   Mogno
## 3   Cedro
## 4  Tauari
```

```
## [1] "Acapu"  "Mogno"  "Cedro"  "Tauari"
```

Indexação no R

Indexação de listas - Níveis superiores nomeados

Lista com níveis superiores nomeados

```
diametro <- c(23.0, 27.0, 33.6, 52.6)

invFlor <-
  data.frame(
    especie=c("Acapu", "Mogno", "Cedro", "Tauari"),
    cortar=c("Não", "Não", "Não", "Sim"),
    stringsAsFactors=FALSE)

# Criando lista não nomeada
(list <- list(DAP = diametro, Inventario = invFlor))

# extrai o componente da posição 1
list$DAP

# extrai a coluna espécie do componente 2
list$Inventario$especie

# extrai as árvores para corte com DAP > 50
list$Inventario[diametro > 50 & cortar == "Sim", ]
```

```
## $DAP
## [1] 23.0 27.0 33.6 52.6
##
## $Inventario
##   especie cortar
## 1   Acapu    Não
## 2   Mogno    Não
## 3   Cedro    Não
## 4  Tauari    Sim
```

```
## [1] 23.0 27.0 33.6 52.6
```

```
## [1] "Acapu"  "Mogno"  "Cedro"  "Tauari"
```

```
##   especie cortar
## 4  Tauari    Sim
```

Parte 4

Um pouco mais sobre o R



Um pouco mais sobre o R

Sintaxe da linguagem R

- **Linguagem de expressão**: Tecnicamente, R é uma linguagem de expressão com uma sintaxe muito simples.
- **Case sensitive**: A linguagem R é *case sensitive*. Isto é, diferencia maiúscula e minúscula.

```
"a" == "A"
"mogno" == "Mogno"

# Cria um vetor de nomes
esp <- c("Cedro", "Ipe", "Ipe", "Ipe", "Cedro",
         "Ipe", "Ipe", "Cedro", "Ipe", "ipe")
table(esp) # tabela de frequências
```

```
## [1] FALSE
## [1] FALSE
## esp
## Cedro   ipe   Ipe
##     3     1     6
```

Fonte: An Introduction to R

Um pouco mais sobre o R

Sintaxe da linguagem R

- **Comandos elementares no R**: consistem em expressões ou atribuições.

Se uma expressão é um comando, então o R: avalia, imprime (a menos que seja especificado ao contrário) e o resultado é "perdido".

Se uma atribuições é um comando, então o R: avalia, atribui o resultado para uma variável e suprime a impressão automática do resultado".

```
# Atribuição: Avalia, atribui e suprime
DAP <- c(49, 56, 44, 46, 56,
         87, 45, 98, 100, 120)

# Expressão: Avalia, imprime e perde o resultad
mean(DAP)
```

```
## [1] 70.1
```

Fonte: An Introduction to R

Um pouco mais sobre o R

Sintaxe da linguagem R

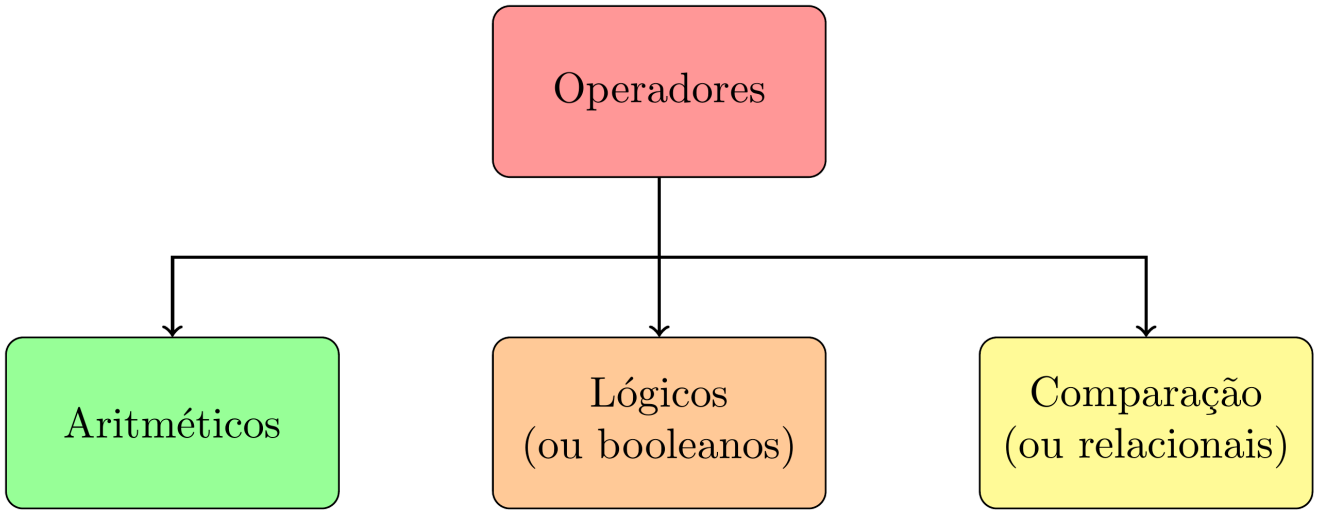
- **Separação de comandos**: ponto e vírgula (;) ou por uma nova linha;
- **Comentários**: precedidos pelo símbolo # (tudo após # não é executado);
- **Comando incompleto**: se o comando R não estiver sintaticamente completo (em uma linha de código), o prompt aparecerá com o símbolo: +. Esse símbolo indica continuação do comando inicial.

Fonte: An Introduction to R

Um pouco mais sobre o R

Operadores no R

- Na linguagem R existe três tipos de operadores
- Os operadores lógicos e relacionais operam com duas respostas possíveis: `TRUE` (verdadeiro) ou `FALSE` (falso).



Um pouco mais sobre o R

Tipos de operadores

Operadores Aritméticos

Símbolo	Descrição
+	Adição
-	Subtração
*	Multiplicação
/	Divisão
^ ou **	Potenciação
%%	Resto da divisão
%/%	Parte inteira da divisão

Operadores Lógicos

Símbolo	Descrição
&	E (and) - versão vetorizada
&&	E (and) - versão não-vetorizada
|	Ou (or) - versão vetorizada
||	Ou (or) - versão não-vetorizada
!	Não...
xor	Ou exclusivo...

Operadores Relacionais

Símbolo	Descrição
<	Menor do que...
>	Maior do que...
<=	Menor ou igual do que...
>=	Maior ou igual do que...
==	Igual a...
!=	Diferente de...
%in%	Contém

Um pouco mais sobre o R

Operadores aritméticos - Expressões básicas

- Operadores usuais para realizar operações matemáticas.

```
2+3             # adição
4*9             # multiplicação
20/5            # divisão
32-10           # subtração
5^3             # potenciação (exponenciação)
10%%3           # resto da divisão
10%/%3          # Parte inteira da divisão
```

```
## [1] 5
## [1] 36
## [1] 4
## [1] 22
## [1] 125
## [1] 1
## [1] 3
```

```
(x <- c(2,4,6,8))
(y <- c(1,5,9,0))
```

```
## [1] 2 4 6 8
## [1] 1 5 9 0
```

```
x + y
y/x+2
```

```
## [1]  3  9 15  8
## [1] 2.50 3.25 3.50 2.00
```

Acesse o script: 01-R-Operadores

Um pouco mais sobre o R

Operadores aritméticos - Expressões mais complexas

- Regra **PEMDAS** (**P**arênteses, **E**xponenciação, **M**ultiplicação, **D**ivisão, **A**dição e **S**ubtração).

```
(2+3)*10
4*3**3
5+8-4*9/3
5+(8-4)*9/3
2^3*4+6/2
2^3*(4+6)/2
```

```
## [1] 50
## [1] 108
## [1] 1
## [1] 17
## [1] 35
## [1] 40
```

Um pouco mais sobre o R

Operadores relacionais (ou comparação)

- Operadores relacionais são usados para realizar comparações de valores.

```
# vetor com um elemento
(x <- 20)
(y <- 10)
(v <- 1)

# vetor com mais de um elemento
(z <- c(1:5))
(w <- c(5:1))
```

```
## [1] 20
## [1] 10
## [1] 1
## [1] 1 2 3 4 5
## [1] 5 4 3 2 1
```

```
x > y         # x é maior do que y?
x < y         # x é menor do que y?
x != y        # x é diferente de y?
y <= x        # y é menor ou igual a x?
y >= x        # y é maior ou igual a x?
z == w        # elementos de z são iguais aos de w?
z != w        # elementos de z são dif. aos de w?
x >= w        # x é >= aos elementos de w?
w %in% v      # w contém v?
```

```
## [1] TRUE
## [1] FALSE
## [1] TRUE
## [1] TRUE
## [1] FALSE
## [1] FALSE FALSE  TRUE FALSE FALSE
## [1]  TRUE  TRUE FALSE  TRUE  TRUE
## [1] TRUE TRUE TRUE TRUE TRUE
## [1] FALSE FALSE FALSE FALSE  TRUE
```

Um pouco mais sobre o R

Operadores relacionais (ou comparação) - operador %in%

É um operador binário que retorna um vetor booleano (TRUE ou FALSE) de tamanho sempre igual ao vetor esquerdo.

Considere um vetor de nomes de espécies florestais...

```
especie <-
   c("Vouacapoua-americana", "Cedrela-odorata",
     "Bertholletia-excelsa", "Dinizia-excelsa Ducke",
     "Bertholletia-excelsa", "Manilkara-huberi",
     "Couratari-guianensis")

"Bertholletia-excelsa" %in% especie
"Swietenia-macrophylla" %in% especie
```

```
## [1] TRUE
## [1] FALSE
```

Agora, você deseja descobrir se esse vetor contém *Bertholletia excelsa* e *Swietenia macrophylla*?

Essas espécies estão classificadas na categoria Vulnerável (VU), conforme a Portaria MMA 443, de 17 de dezembro de 2014. Portanto, sofrem as restrições estabelecidas no art. 2º, incluindo a proibição de manejo.

Um pouco mais sobre o R

Operadores relacionais (ou comparação)

```
especie <-
   c("Vouacapoua-americana", "Cedrela-odorata",
     "Bertholletia-excelsa", "Dinizia-excelsa Ducke",
     "Bertholletia-excelsa", "Manilkara-huberi",
     "Couratari-guianensis")

ameacadas <-
  c("Vouacapoua-americana", "Bertholletia-excelsa",
    "Cedrela-odorata", "Swietenia-macrophylla")

especie %in% ameacadas
#unique(especie[which(especie %in% ameacadas)])
```

```
## [1]  TRUE  TRUE  TRUE FALSE  TRUE FALSE FALSE
```

A pergunta feita é:

- As espécies do vetor "ameacadas" estão contidas no vetor "especie"?
- O comando **%in%** avalia cada elemento do vetor "especie" e retorna uma resposta booleana (TRUE ou FALSE) de comprimento igual ao vetor esquerdo ("especie").

Um pouco mais sobre o R

Operadores lógicos

- Em programação, estes operadores são usados para realizar **operações lógicas**, cuja saída é uma **resposta booleana**.
- Uma resposta booleana indica se algo é Verdadeiro (`TRUE`) ou Falso (`FALSE`).

Mogno brasileiro = *Swietenia macrophylla*

Angelim pedra = *Hymenolobium petraeum*

Dados fictícios para 10 árvores de duas espécies florestais.

Especie	DAP	H	Cipo	QF	Selecao
Swietenia macrophylla	33.6	9.1	Sim	2	Protegida
Swietenia macrophylla	42.6	13.2	Não	1	Protegida
Swietenia macrophylla	52.1	15.4	Sim	3	Protegida
Hymenolobium petraeum	80.3	18.6	Não	1	Explorar
Hymenolobium petraeum	90.8	19.5	Sim	2	Explorar
Hymenolobium petraeum	49.4	16.8	Não	1	Remanescente
Hymenolobium petraeum	70.5	17.9	Sim	2	Explorar
Hymenolobium petraeum	100.5	22.4	Não	3	Remanescente

Um pouco mais sobre o R

Operadores lógicos

Operador & - versão vetorizada

Use o operador lógico & para responder as questões a seguir:

👉 Quais árvores possuem **DAP maior ou igual a 50cm** e **Qualidade de Fuste (QF) igual a 1** ❓

```
data$DAP >= 50 & data$QF == 1
#data[data$DAP >= 50 & data$QF == 1, ]
```

```
##  [1] FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE  TRUE FALSE
```

🌳 Legislação florestal! 🌳

A Resolução CONAMA nº 406 de 02/02/2009 (Publicada no DOU nº 26, de 06/02/2009, pág. 100) estabelece em seu art. 6º um **Diâmetro Mínimo de Corte (DMC) de 50 cm** para todas as espécies, para as quais ainda não se estabeleceu o DMC específico.

Um pouco mais sobre o R

Operadores lógicos

Operador & - versão vetorizada (cont.)

👉 Quais árvores possuem **cipós ausentes**, **QF igual a 1** e **DAP maior ou igual a 50cm** ❓

```
data$Cipo == "Não" & data$QF == 1 & data$DAP >= 50
#data[data$Cipo == "Não" & data$QF == 1 & data$DAP >= 50,]
```

```
##  [1] FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE
```

👉 Quais árvores estão selecionadas para **explorar** e **QF igual a 1** ❓

```
data$Selecao == "Explorar" & data$QF == 1
#data[data$Selecao == "Explorar" & data$QF == 1, ]
```

```
##  [1] FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE
```

Um pouco mais sobre o R

Funções matemáticas usuais

A linguagem R dispõe de funções matemáticas usualmente encontradas em calculadoras científicas.

Função	Descrição
sqrt()	Raiz quadrada
abs()	Valor absoluto
sin(); cos(); tan()	Funções trigonométricas
asin(); acos(); atan()	Funções trigonométricas inversas
exp()	Exponencial
log10()	Logarítmo na base 10
log()	Logarítmo natural
factorial	Fatorial

Um pouco mais sobre o R

Alguns comandos básicos

Comando	Ação
q()	Fechar o programa
rm(nome do objeto)	Remover um objeto qualquer
ls()	Listar os objetos na janela de trabalho atual
help(nome da função) ou ?nome da função	Solicitar ajuda sobre o uso de uma função
save.image()	Salvar
Ctrl + L	Limpar a tela do R console
history(max.show, nrow = 3)	Listar os últimos 3 comandos executados
getwd()	Mostrar o diretório de trabalho
setwd("diretório desejado")	Mudar o diretório de trabalho
install.packages("nome do pacote")	Instalar um pacote específico

Um pouco mais sobre o R

Alguns comandos básicos (cont.)

Comando	Ação
library("nome do pacote")	Carregar um pacote específico
dir()	Lista os arquivos existentes no diretório
getOption("OutDec")	Verificar o separador decimal definido
options("OutDec=")	Mudar o separador decimal para vírgula
round(5.9845, digits=2)	Função para arredondamento de casas decimais
data()	Lista de conjuntos de dados disponíveis no R
?nomedodataset	Obter informações detalhadas sobre um conjunto de dado
class(nome do objeto)	Verifica a classe de um objeto específico
search()	Lista todos os pacotes carregados

Até a Próxima!

LMFTCA
LABORATÓRIO DE MANEJO FLORESTAL, TECNOLOGIAS E COMUNIDADES AMAZÔNICAS
ENGENHARIA FLORESTAL
UFPA

👫 @lmftca_ufpa

🌎 https://www.lmftca.com.br/